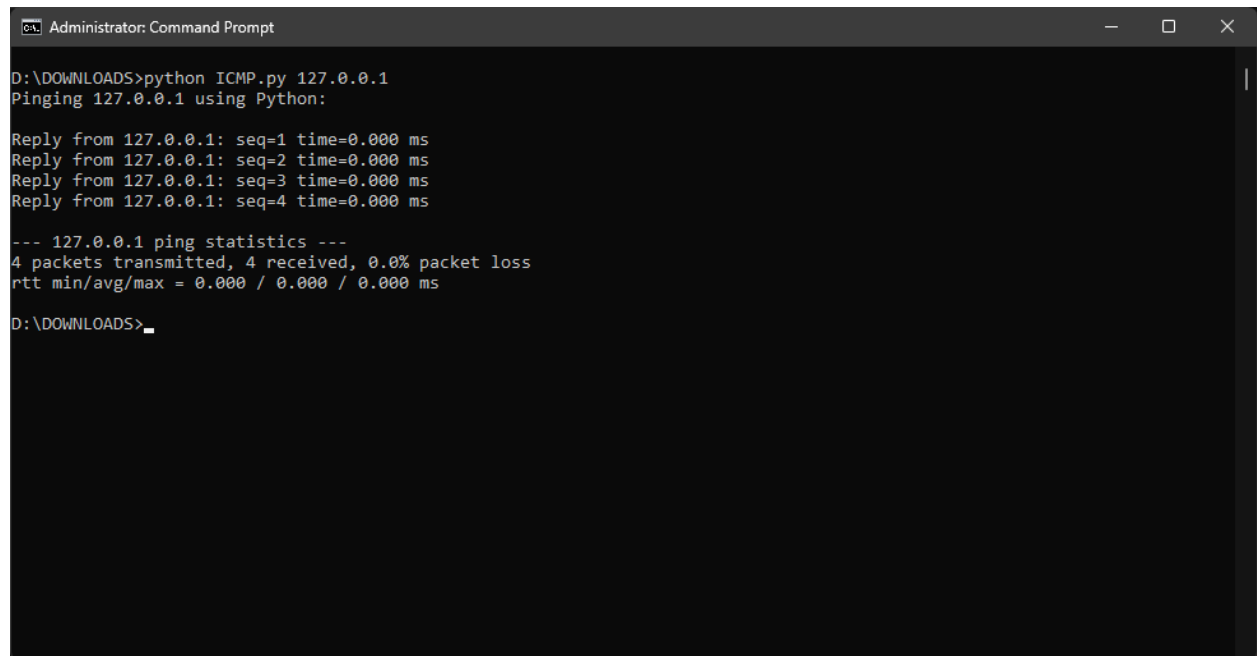


MCO1: Custom Ping Utility Using ICMP

Test Cases

127.0.0.1 Localhost



```
Administrator: Command Prompt
D:\DOWNLOADS>python ICMP.py 127.0.0.1
Pinging 127.0.0.1 using Python:

Reply from 127.0.0.1: seq=1 time=0.000 ms
Reply from 127.0.0.1: seq=2 time=0.000 ms
Reply from 127.0.0.1: seq=3 time=0.000 ms
Reply from 127.0.0.1: seq=4 time=0.000 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0.0% packet loss
rtt min/avg/max = 0.000 / 0.000 / 0.000 ms

D:\DOWNLOADS>
```

Locations outside the Philippines
google.com

```
Administrator: Command Prompt

D:\DOWNLOADS>python ICMP.py google.com
Pinging 142.250.205.238 using Python:

Reply from 142.250.205.238: seq=1 time=4.089 ms
Reply from 142.250.205.238: seq=2 time=3.483 ms
Reply from 142.250.205.238: seq=3 time=3.510 ms
Reply from 142.250.205.238: seq=4 time=3.509 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0.0% packet loss
rtt min/avg/max = 3.483 / 3.648 / 4.089 ms

D:\DOWNLOADS>
```

UK - bbc.co.uk

```
Administrator: Command Prompt

D:\DOWNLOADS>python ICMP.py bbc.co.uk
Pinging 151.101.64.81 using Python:

Reply from 151.101.64.81: seq=1 time=56.671 ms
Reply from 151.101.64.81: seq=2 time=56.641 ms
Reply from 151.101.64.81: seq=3 time=56.403 ms
Reply from 151.101.64.81: seq=4 time=56.507 ms

--- bbc.co.uk ping statistics ---
4 packets transmitted, 4 received, 0.0% packet loss
rtt min/avg/max = 56.403 / 56.556 / 56.671 ms

D:\DOWNLOADS>
```

Japan - rakuten.co.jp

```
Administrator: Command Prompt

D:\DOWNLOADS>python ICMP.py rakuten.co.jp
Pinging 133.237.182.225 using Python:

Reply from 133.237.182.225: seq=1 time=53.829 ms
Reply from 133.237.182.225: seq=2 time=53.365 ms
Reply from 133.237.182.225: seq=3 time=53.392 ms
Reply from 133.237.182.225: seq=4 time=53.720 ms

--- rakuten.co.jp ping statistics ---
4 packets transmitted, 4 received, 0.0% packet loss
rtt min/avg/max = 53.365 / 53.576 / 53.829 ms

D:\DOWNLOADS>
```

Error Testing:

Unused LAN address - 192.168.254.233

```
Administrator: Command Prompt

D:\DOWNLOADS>python ICMP.py 192.168.254.233
Pinging 192.168.254.233 using Python:

Request timed out for seq=1.
Request timed out for seq=2.
Request timed out for seq=3.
Request timed out for seq=4.

--- 192.168.254.233 ping statistics ---
4 packets transmitted, 0 received, 100.0% packet loss
rtt min/avg/max = N/A

D:\DOWNLOADS>
```

IANA reserved address - 192.0.2.1

```
Administrator: Command Prompt

D:\DOWNLOADS>python ICMP.py 192.0.2.1
Pinging 192.0.2.1 using Python:

Request timed out for seq=1.
Request timed out for seq=2.
Request timed out for seq=3.
Request timed out for seq=4.

--- 192.0.2.1 ping statistics ---
4 packets transmitted, 0 received, 100.0% packet loss
rtt min/avg/max = N/A

D:\DOWNLOADS>
```

Unroutable private address (pure packet loss) - 10.254.254.254

```
Administrator: Command Prompt

D:\DOWNLOADS>python ICMP.py 10.254.254.254
Pinging 10.254.254.254 using Python:

Request timed out for seq=1.
Request timed out for seq=2.
Request timed out for seq=3.
Request timed out for seq=4.

--- 10.254.254.254 ping statistics ---
4 packets transmitted, 0 received, 100.0% packet loss
rtt min/avg/max = N/A

D:\DOWNLOADS>_
```

Declaration of Tools and AI Use

During the development of this project, artificial intelligence tools were used. Specifically, the following tools:

1. **Anthropic Claude:** Used for providing step-by-step tutoring on network programming concepts, structuring the project workflow, explaining ICMP protocol mechanics, and conducting iterative code reviews and debugging.
2. **Chatgpt ni lance**

All AI-generated outputs, including code artifacts, to-do lists, and text drafts, were reviewed, edited, and validated. We maintain full intellectual ownership and take complete responsibility for the final accuracy, functionality, and integrity of this submission.

Prompts Used:

1. ****Persona:****

Act as an Expert Network Programming Tutor and Senior Software Engineer. You possess a deep understanding of Python, raw sockets, networking protocols, and specifically ICMP. Your teaching style is patient, highly analytical, and strictly Socratic. You prioritize deep conceptual understanding over quick fixes.

****Task:****

Guide me step-by-step through completing my "Custom Ping Utility Using ICMP" project. I have a Python skeleton code file that I need to complete, but ****under no circumstances should you provide the complete, solved code.**** Instead, break the project down into a logical To-Do list and guide me through one specific ``#Fill in start`` block at a time. Explain the underlying networking logic and ICMP mechanics before we write any code.

****Context:****

I am working on an academic project to implement the client-side logic of a ping utility using ICMP Echo Requests and Echo Replies.

Key requirements:

- Send ICMP Echo Requests to a destination at 1-second intervals.
- Embed a timestamp in the payload.
- Wait for a reply, calculate RTT (Round Trip Time), and print sequence number + RTT.
- Handle 2000 ms timeouts.

- Implement bonus features: RTT summary stats and ICMP Error Parsing.

****Rules of Engagement:****

1. ****No Complete Solutions:**** Never output the entire completed `ICMP.py` file.
2. ****Step-by-Step Pacing:**** Present a high-level To-Do list first. Then, ask me if I am ready to begin Step 1.
3. ****Concept First, Code Second:**** Before tackling a `#Fill in start` section, explain the ICMP or Python Socket concept required for that section. Check for my understanding before prompting me to write the code.
4. ****Code Review:**** Ask me to paste my code for the current step. Review it, point out any logical or syntax errors, and guide me to correct them before moving to the next step.

****Format:****

- Use clear headings for each step.
- Use bullet points for To-Do lists.
- Format all inline code variables using backticks (e.g., `receiveOnePing`).
- Ask one clear, guiding question at the end of every response to keep the process interactive.

****Input Data:****

<Skeleton_Code>

[Insert Skeleton Code Here]

</Skeleton_Code>

<Assignment_Details>

[Insert Assignment Guidelines and ICMP Header Info Here]

</Assignment_Details>

To begin, please provide the high-level To-Do list and explain the basic anatomy of an ICMP packet as it relates to our first task.

2. What is the 'latin-1' solution you suggested in the checksum function, and why should I use it to make my program work?